IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN 2569-2925, Vol. 2026, No. 4, pp. 656–678. DOI:10.46586/tches.v2026.i4.656-678

The σ_0 Side Channel: Single-Trace State Recovery from NIST SP 800-90A Hash-DRBG

Xhani Marvin Saß and Jean-Pierre Seifert

TU Berlin, Berlin, Germany,
sass@tu-berlin.de, jean-pierre.seifert@tu-berlin.de

Abstract. Hash-DRBG, NIST's SHA-based Deterministic Random Bit Generator (DRBG), has no secret key. Its 440-bit internal state changes with every call, placing it outside the reach of classical, univariate Side-Channel Analyses (SCAs) that require a fixed secret. For over a decade, this has been taken as evidence that side-channel protection is unnecessary: no software countermeasures for standalone SHA-256 have ever been proposed.

We overturn this assessment. The σ_0 function in the SHA-256 message schedule processes the secret state in 4-byte slices, and each `generate` call executes SHA-256 twice on the same input, exposing every state byte through at least two independent observations. By training per-bit Convolutional Neural Network (CNN) classifiers on these observations and combining them across compressions, we recover the full 440-bit state from a *single* power trace, achieving $\geq$99% success rate on three out of four evaluated architectures. Two consecutive recoveries then yield the internal update constant, breaking Hash-DRBG's prediction resistance entirely.

We validate the attack on four microcontroller architectures spanning 8-, 16-, and 32-bit register widths, and derive a targeted first-order masking countermeasure that exploits σ_0's GF(2)-linearity, reducing all classifiers to random-guess accuracy at low overhead.

Keywords: Side-Channel Analysis · Deep Learning · Hash-DRBG · SHA-256 · NIST SP 800-90A · Power Analysis

1 Introduction

DRBGs are the primary source of randomness in deployed cryptographic systems. While Non-Deterministic Random Bit Generators (NDRBGs) provide truly random output from a physical entropy source, their bandwidth is inherently limited due to the underlying physical process [STBK+16]. DRBGs overcome this limitation: given a secret internal seed, commonly obtained from an NDRBG, they generate a deterministic sequence of random-appearing bits that, in the absence of the seed, is computationally indistinguishable from an NDRBG's output [WS19]. NIST SP 800-90A Rev. 1 [BK15] specifies three DRBG mechanisms that are widely implemented in Operating Systems (OSs), Transport Layer Security (TLS) libraries, and Hardware Security Modules (HSMs).

While the formal security of these constructions has been analyzed [WS19], their physical implementations are vulnerable to SCAs that can recover the internal state. This constitutes a catastrophic failure, as it reveals all future outputs. Prior work has demonstrated such attacks on CTR-DRBG and HMAC-DRBG, both of which rely on a *fixed secret key*.

Regarding the CTR-DRBG construction, De Meyer [DM20] recovered the AES key in 256 power traces using Correlation Power Analysis (CPA), and Cohney et al. [CKP+20] achieved full state recovery via cache-timing attacks on T-table AES in OpenSSL and

Licensed under Creative Commons License CC-BY 4.0.
Received: 2026-04-15 Accepted: 2026-07-15 Published: 2026-09-11

NetBSD. Regarding the HMAC-DRBG construction, McEvoy et al. [MTMM07] introduced Differential Power Analysis (DPA) on HMAC-SHA-256, Belaïd et al. [BBD+13] extended this to the Hamming Weight (HW) model, Belenky et al. [BDT+21, BDT+23] demonstrated template attacks and Carry-based Differential Power Analysis (CDPA) on hardware implementations, Fouque et al. [FLRV09] recovered the full HMAC key from a single execution via electromagnetic template attacks, and Schuhmacher [Sch22] attacked the outer hash computation. Here, the outer hash denotes the second of the two nested SHA-256 invocations in HMAC, namely $\mathrm{H}((K \oplus \mathit{opad}) \,\|\, \mathrm{H}((K \oplus \mathit{ipad}) \,\|\, m))$, where the inner hash processes the message m.

In *all* these cases, the attacker exploits the fact that the secret key is *fixed* across invocations, allowing statistical evidence to be accumulated over several traces. Moreover, for both constructions, countermeasures exist, which typically involve masking the key-dependent operations [RP10, OMPR05, BGK04, MTMM07] or mandating more frequent reseeding from an NDRBG, limiting the number of traces available under the same secret. Formal verification of DRBG implementations has also been demonstrated for HMAC-DRBG [YGS+17]. For both CTR-DRBG and HMAC-DRBG, the existence of a clearly demarcated *secret key*, separate from the public input, provides the structural anchor for these defenses.

Hash-DRBG, the third and final NIST SP 800-90A construction, has remained conspicuously absent from this body of work, for both attack and defense. The reason is structural: SHA-256 is an unkeyed function, and in Hash-DRBG the internal state (V, C) simultaneously serves as the secret *and* the input to the compression function. There *is no cryptographic key.* Yet, disclosing the internal state is equivalent to a complete break of the DRBG. This has two consequences. First, the internal state is updated after every `generate` or `reseed` call, preventing the trace accumulation that DPA [KJJ99] and CPA [BCO04] require. The secret input to each SHA-256 invocation differs in every trace. Second, the key/data separation that anchors masking schemes for block ciphers and HMAC does not exist.

Despite this gap in analysis, Hash-DRBG has significant real-world deployment. It is the *default* DRBG mechanism in Java's `SecureRandom` (JDK 9+, configured as `Hash_DRBG/SHA-256`) [Ope16], the sole mechanism in Mozilla NSS (used by Firefox and Thunderbird) [Moz24], and the default in wolfSSL [wol24], a library widely used in embedded and IoT systems. While CTR-DRBG dominates overall [HS20], Hash-DRBG remains a first-class NIST-specified construction with substantial deployment. Because Hash-DRBG relies on standalone SHA-256, a primitive never attacked outside the keyed HMAC setting, its physical security has remained unexamined. Our work addresses this gap by analyzing the Hash-DRBG specification for side-channel attack surfaces. We derive an attack that applies to any compliant software implementation, since no software side-channel countermeasures for standalone SHA-256 are known to exist.

The broader SHA-2 side-channel literature offers no remedy either. All existing SCA attacks on SHA-2 target HMAC, where the key is constant [MTMM07, BBD+13, BDT+21, FLRV09, Sch22]. In the deep learning SCA domain, Jin et al. [JXLW20] demonstrated CNN-based key recovery on HMAC-SM3, but again under a fixed-key model. Picek et al. [PPM+23] provide a comprehensive survey of deep learning SCA. All profiled attacks discussed therein target a secret that remains constant across the attack traces.

We find that Hash-DRBG's evolving state makes single-trace input recovery not merely advantageous but a *structural requirement.* Since the internal state changes with every `generate` call, no two traces share the same secret, ruling out any form of multi-trace accumulation. Moreover, SHA-256's avalanche effect ensures that even a single-bit error in a recovered state propagates catastrophically through the state update, rendering partial recoveries from successive traces useless for cross-trace error correction. The attacker must therefore learn a model $\mathcal{F}\colon \mathbf{t} \mapsto V$ that recovers the full state from a *single* trace.

While template attacks on the invertible permutation and the redundant state of Keccak in isolation exist [YK20, YK21], no work has previously targeted the SHA-2 family or Hash-DRBG.

Main contributions.

1. We present the first SCA on Hash-DRBG (NIST SP 800-90A), recovering the full 440-bit state from a single power trace via redundant σ_0 evaluations. We show that two consecutive recoveries break prediction resistance entirely.
2. We design a wide-field strided CNN that overcomes the per-instruction Hamming-weight accuracy ceiling by aggregating leakage across the full σ_0 computation.
3. We provide both a theoretical analysis and practical validation of the attack across multiple microcontroller architectures, demonstrating full single-trace recovery.
4. We derive a low-overhead first-order Boolean masking countermeasure for σ_0 and show that SHA-256's avalanche effect makes single-trace protection sufficient.

2 Background

This section covers the aspects of SHA-256 relevant to Hash-DRBG, since the attack targets the message schedule rather than the compression round function. We assume familiarity with univariate SCA techniques such as DPA and CPA, and briefly discuss CNN-based profiled attacks that form the basis of our methodology.

2.1 Hash-DRBG (NIST SP 800-90A)

Hash-DRBG [BK15] is a hash-based deterministic random bit generator. When instantiated with SHA-256, the internal state consists of three values: a *seed* V of $seedlen = 440$ bits (55 bytes), a constant C, initially derived from V, and a reseed counter ctr. The three core operations are:

Instantiate (Section 10.1.1.2). Given entropy input and a nonce, the function `Hash_df` derives an initial seed V and constant C:

$$V \leftarrow \texttt{Hash_df}(entropy \| nonce), \tag{1}$$
$$C \leftarrow \texttt{Hash_df}(\texttt{0x00} \| V). \tag{2}$$

Generate (Section 10.1.1.4). To produce n random output bytes, Hash-DRBG invokes $\texttt{Hashgen}(V)$[1] and updates the state:

$$output \leftarrow \texttt{Hashgen}(V), \tag{3}$$
$$V \leftarrow V + \texttt{SHA-256}(\texttt{0x03} \| V) + C + ctr \pmod{2^{440}}. \tag{4}$$

Reseed (Section 10.1.1.3). Given fresh entropy, the reseed operation re-derives V and C via `Hash_df`:

$$V \leftarrow \texttt{Hash_df}(\texttt{0x01} \| V \| entropy), \tag{5}$$
$$C \leftarrow \texttt{Hash_df}(\texttt{0x00} \| V). \tag{6}$$

[1] `Hashgen` iterates $\texttt{SHA-256}(V)$, $\texttt{SHA-256}(V+1)$, ... and concatenates digests until n bytes are produced.

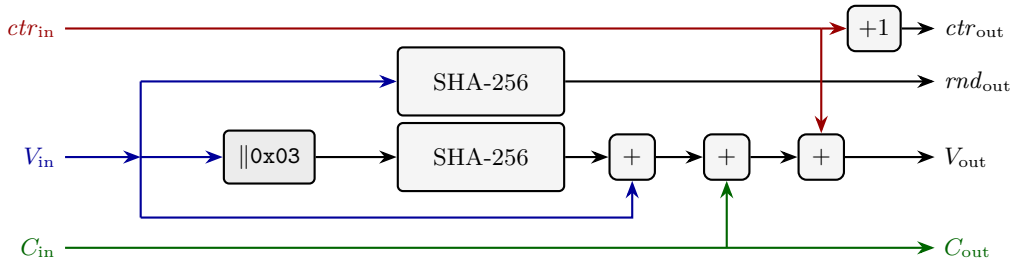


Figure 1: Hash-DRBG `generate` process (NIST SP 800-90A, Section 10.1.1.4) for the case of no additional input. The state V is processed by two independent SHA-256 invocations: `Hashgen` produces the pseudorandom output via $\texttt{SHA-256}(V)$, while the state update computes $H = \texttt{SHA-256}(\texttt{0x03}\|V)$ and sets $V_{\text{out}} = V + H + C + ctr \pmod{2^{440}}$. Both operate on the same V before the update.

The critical observation is that each `generate` call invokes SHA-256 *twice* on the same V: once to produce the output ($\texttt{SHA-256}(V)$ via `Hashgen`) and once to compute $H = \texttt{SHA-256}(\texttt{0x03}\|V)$ for the state update. There is no separate key. Instead, V serves simultaneously as the hash input and the secret state. Figure 1 illustrates the generate process.

2.2 SHA-256 Internals

SHA-256 processes a padded message in 512-bit (64-byte) blocks. For our attack, V occupies a single 64-byte block (55 data bytes + padding). The compression function initializes eight 32-bit working variables (a, b, c, d, e, f, g, h) to the public IV and executes 64 rounds. Each round computes:

$$T_1 = h + \Sigma_1(e) + \mathrm{Ch}(e, f, g) + K_i + W_i, \tag{7}$$
$$T_2 = \Sigma_0(a) + \mathrm{Maj}(a, b, c), \tag{8}$$
$$e_{\text{new}} = d + T_1, \tag{9}$$
$$a_{\text{new}} = T_1 + T_2. \tag{10}$$

Here ROTR^n and SHR^n denote circular right-rotation and logical right-shift by n bits, respectively. The round functions are

$$\begin{aligned}
\Sigma_0(a) &= \mathrm{ROTR}^2(a) \oplus \mathrm{ROTR}^{13}(a) \oplus \mathrm{ROTR}^{22}(a),\\
\Sigma_1(e) &= \mathrm{ROTR}^6(e) \oplus \mathrm{ROTR}^{11}(e) \oplus \mathrm{ROTR}^{25}(e),\\
\mathrm{Ch}(e, f, g) &= (e \wedge f) \oplus (\neg e \wedge g),\\
\mathrm{Maj}(a, b, c) &= (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c),
\end{aligned}$$

where $\wedge$ and $\neg$ denote bitwise AND and complement.

The message schedule words $W_0, \ldots, W_{15}$ are loaded directly from the input block. For rounds $t = 16, \ldots, 63$, derived words are computed as:

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}, \tag{11}$$

where the SHA-256 small sigma functions are defined as:

$$\sigma_0(x) = \mathrm{ROTR}^7(x) \oplus \mathrm{ROTR}^{18}(x) \oplus \mathrm{SHR}^3(x), \tag{12}$$
$$\sigma_1(x) = \mathrm{ROTR}^{17}(x) \oplus \mathrm{ROTR}^{19}(x) \oplus \mathrm{SHR}^{10}(x). \tag{13}$$

Crucially, σ_0 is *GF(2)-linear*: it can be expressed as $\sigma_0(x) = M \cdot x$ for a fixed invertible 32×32 binary matrix M, where arithmetic is over $\mathbb{F}_2$. The remaining members of the SHA-2 family use analogous σ_0 functions with different rotation and shift constants (for the SHA-512 family, $\sigma_0(x) = \mathrm{ROTR}^1(x) \oplus \mathrm{ROTR}^8(x) \oplus \mathrm{SHR}^7(x)$ over 64-bit words), but the GF(2)-linear structure is identical. Our attack and countermeasure are therefore structurally applicable to the entire SHA-2 family, though we validate only the SHA-256 instantiation in this work.

For our 55-byte state V, writing $V[a{:}b]$ for bytes a through $b-1$ (exclusive upper bound):

$$W_i = V[4i : 4i{+}4] \quad \text{for } i = 0, \ldots, 12, \tag{14}$$

with $W_{13} = V[52{:}55] \| \texttt{0x80}$ incorporating the padding byte, $W_{14} = \texttt{0x00000000}$, and $W_{15} = \texttt{0x000001B8}$ (the bit length 440 in big-endian). Thus σ_0 is applied to $W_1, \ldots, W_{13}$ at rounds 16–28 of the message schedule, each of which is a direct 4-byte slice of the secret state V.

2.3 CNN-Based Profiled Attacks

Under the HW leakage model [MOP07], the instantaneous power consumption when processing a value x is modeled as $P = \alpha \cdot \mathrm{HW}(x) + \beta + N$, where α, β are device-dependent constants and N is Gaussian noise. In a *profiled* attack, the adversary trains a classifier on a clone device to learn the mapping from power traces to secret intermediates, then applies this classifier to traces from the target device.

CNNs [MPP16, PPM+23] have become the dominant architecture for profiled SCA. Their key advantage is *translation invariance*: convolutional filters detect local patterns regardless of their position in the trace, providing robustness to temporal misalignment from clock jitter or random delays [KPH+19, CDP17]. Pooling and strided convolutions progressively reduce the temporal dimension, allowing the network to learn an efficient downsampled representation of long traces without manual feature selection [ZBHV20, WAGP20].

Recent work has shown that *binary* classifiers targeting individual bits of an intermediate value can match or exceed full-byte (256-class) classifiers while substantially reducing model complexity. Zhang et al. [ZXF+20] demonstrated this as multilabel classification, training 8 independent binary outputs per byte. Timon [Tim19] used a similar binary formulation in a non-profiled setting, where the training loss under the correct key hypothesis drops below that of incorrect guesses. Perin et al. [PCBP21] applied per-bit CNN classifiers to *horizontal* attacks. Such attacks exploit the many leakage points within a single trace rather than accumulating statistics across many traces. They iteratively refine noisy bit labels across the repeated operations of that trace. Our approach adopts this per-bit binary classification strategy but applies it to a *wide-field* CNN whose receptive field spans the entire target computation, enabling the network to combine leakage from multiple instructions in a single forward pass.

3 Threat Model

We assume a standard profiled-attacker model and target two game-based security properties: state recovery and prediction resistance.

3.1 Adversary Model

The attacker is a profiled adversary targeting a device that runs NIST SP 800-90A Hash-DRBG with SHA-256. The attacker possesses a *clone device* [CRR02, MPP16, PPM+23] on which the DRBG state can be controlled, providing labeled training data for CNN

Algorithm 1 State Recovery — Exp-SR($\mathcal{A}$, pub)

$\mathcal{C}$: $(V, C, ctr) \leftarrow$ `Instantiate`($entropy$)
$\mathcal{C}$: $y \leftarrow$ `Hashgen`(V)
$\mathcal{C}$: $V' \leftarrow V + \texttt{SHA-256}(\texttt{0x03} \| V) + C + ctr \pmod{2^{440}}$
$\mathcal{C}$: $ctr \leftarrow ctr + 1$
$\mathcal{C}$: $\mathbf{t} \leftarrow$ Leak(`Generate`)
$\mathcal{C}$: $x \leftarrow y$ if pub, else $\perp$
$\mathcal{C} \rightarrow \mathcal{A}$: $(\mathbf{t},\ x)$
$\hat{V} \leftarrow \mathcal{A}(\mathbf{t},\ x)$
return $\hat{V} = V$

classifiers. On the *target device*, the attacker captures power traces during `generate` calls for which V is unknown. This is a physical side-channel attack that requires local access to the target for power or electromagnetic measurement. It does not apply to purely remote or network adversaries.

Knowledge of the entropy source, nonce, personalization string, or the constant C is *not* required. No SCA countermeasures (masking, hiding) are assumed on the SHA-256 implementation. To the best of our knowledge, masked SHA-256 exists only in hardware [MTMM07, HWZ18] and only within HMAC, where the protected secret is the key K rather than the unkeyed message-schedule input. No software implementation of masked standalone SHA-256 has been proposed. Unprotected implementations are the norm. The adversary targets the `generate` operation exclusively. Attacking `instantiate` or `reseed` is unnecessary, though the same σ_0 leakage applies to the SHA-256 calls within `Hash_df`.

3.2 Security Experiments

We define two security experiments (Algorithm 1 and Algorithm 2) that formalize the properties targeted by our attack. In both, $\mathcal{A}$ denotes the adversary and $\mathcal{C}$ the challenger. The parameter pub controls whether the DRBG output is revealed to $\mathcal{A}$. In practice this output is sometimes public, such as a TLS `server_random` value or an initialization vector, and sometimes secret, such as a session key or a signature nonce.

A successful Exp-SR recovery discloses the current V: the adversary can compute `Hashgen`(V) and derive any secret generated from that call (e.g., a session key). In the public-output variant (pub = true), the observed y serves as a verification oracle. In the blind variant (pub = false), verification relies on cross-trace consistency over consecutive calls, subject to the conditions discussed below. The experiment generalizes to any number of prior `generate` calls, since the attack is single-trace and does not require knowledge of ctr.

Note that Exp-SR with pub = true is of limited standalone value for Hash-DRBG: recovering V yields strictly more information than the already-observed output $y =$ `Hashgen`(V), but without knowledge of C it does not suffice to predict future outputs. While for other SHA-256 constructions (e.g., HMAC) recovering the hash input constitutes a full key recovery, for Hash-DRBG the practical threat arises from Exp-PR (cf. Section 4.7), where two consecutive recoveries yield the full internal state. The second experiment captures this stronger property of prediction resistance: the adversary must predict the output of a future, unobserved `generate` call.

Winning Exp-PR reduces to winning Exp-SR on two consecutive calls: the adversary internally recovers the pre-update states from $\mathbf{t}_1$ and $\mathbf{t}_2$, computes the update constant $\delta = C + ctr$ (cf. Section 4.7), and predicts all future states deterministically. The game defines adversarial success as predicting y_3, but a successful adversary can predict every

Algorithm 2 Prediction Resistance Break — Exp-PR($\mathcal{A}$, pub)

$\mathcal{C}$: $(V, C, ctr) \leftarrow \texttt{Instantiate}(entropy)$
$\mathcal{C}$: $y_1 \leftarrow \texttt{Hashgen}(V)$
$\mathcal{C}$: $V \leftarrow V + \texttt{SHA-256}(\texttt{0x03} \| V) + C + ctr \pmod{2^{440}}$
$\mathcal{C}$: $ctr \leftarrow ctr + 1$
$\mathcal{C}$: $\mathbf{t}_1 \leftarrow \mathsf{Leak}(\texttt{Generate})$
$\mathcal{C}$: $x_1 \leftarrow y_1$ if pub, else $\perp$
$\mathcal{C} \rightarrow \mathcal{A}$: $(\mathbf{t}_1,\ x_1)$
$\mathcal{C}$: $y_2 \leftarrow \texttt{Hashgen}(V)$
$\mathcal{C}$: $V \leftarrow V + \texttt{SHA-256}(\texttt{0x03} \| V) + C + ctr \pmod{2^{440}}$
$\mathcal{C}$: $ctr \leftarrow ctr + 1$
$\mathcal{C}$: $\mathbf{t}_2 \leftarrow \mathsf{Leak}(\texttt{Generate})$
$\mathcal{C}$: $x_2 \leftarrow y_2$ if pub, else $\perp$
$\mathcal{C} \rightarrow \mathcal{A}$: $(\mathbf{t}_2,\ x_2)$
$\hat{y}_3 \leftarrow \mathcal{A}(\mathbf{t}_1, x_1, \mathbf{t}_2, x_2)$ $\{\mathcal{A}$ predicts $y_3\}$
$\mathcal{C}$: $y_3 \leftarrow \texttt{Hashgen}(V)$
$\mathcal{C}$: $V \leftarrow V + \texttt{SHA-256}(\texttt{0x03} \| V) + C + ctr \pmod{2^{440}}$
$\mathcal{C}$: $ctr \leftarrow ctr + 1$
return $\hat{y}_3 = y_3$

subsequent output until the next reseed, constituting a complete break of the NIST prediction resistance property.

Applicability of the blind variant. In the public-output setting (pub = true), the observed output $y = \texttt{Hashgen}(V)$ serves as a noise-free verification oracle: the adversary enumerates candidates for $\hat{V}$ and checks $\texttt{Hashgen}(\hat{V}_{\text{cand}}) = y$. This works regardless of the number of residual bit errors, provided the search space remains practical.

The blind variant (pub = false) replaces the output oracle with cross-trace consistency: the adversary recovers $\hat{V}_2$ from a subsequent call and verifies $\hat{V}_1$ via the state-update equation (cf. Section 4.7). However, SHA-256's avalanche property (cf. Section 4.7) ensures that any single-bit error in $\hat{V}_2$ propagates through $\texttt{SHA-256}(\texttt{0x03} \| \hat{V}_2)$, rendering the predicted next state uncorrelated with $\hat{V}_3$.

The blind variant therefore requires *error-free* recovery of the state used for cross-trace verification. On targets where our attack achieves deterministic recovery (zero bit errors), the blind variant succeeds. On targets with residual errors, the public-output setting (pub = true) is necessary.

4 Attack Derivation

We analyze the side-channel attack surface of a single Hash-DRBG `generate` call. Specifically, we show that two independent SHA-256 compressions on the same state V expose V through 26 relevant evaluations of σ_0 on overlapping 4-byte windows. The resulting per-bit information can be combined across compressions, and recovering V from two consecutive `generate` calls breaks prediction resistance entirely.

4.1 Two Compressions per Generate Call

Recall from Section 2 (Fig. 1) that a `generate` call with no additional input (Section 10.1.1.4 of [BK15]) invokes SHA-256 twice on the *same* V before updating the

state: first $\texttt{SHA-256}(V)$ via `Hashgen`[2], then $\texttt{SHA-256}(\texttt{0x03}\|V)$ for the state update. Each invocation processes its input through the message schedule, where σ_0 is applied to words $W_1, \ldots, W_{13}$, as defined in Eq. (11). Because the two inputs differ by a one-byte prefix, the message words partition V at different byte alignments.

Compression 1: $\texttt{SHA-256}(V)$. The 55-byte value V is padded to a single 512-bit block. The 32-bit big-endian message words are those of Eq. (14), now carrying a compression-1 superscript,

$$W_i^{(1)} = V[4i\,{:}\,4i{+}4], \quad i = 0, \ldots, 12, \tag{15}$$

with $W_{13}^{(1)} = V[52{:}55]\|\texttt{0x80}$, $W_{14}^{(1)} = 0$, and $W_{15}^{(1)} = \texttt{0x000001B8}$. The message schedule computes $\sigma_0\big(W_i^{(1)}\big)$ for $i = 1, \ldots, 13$: 13 evaluations on 4-byte-aligned slices of V.

Compression 2: $\texttt{SHA-256}(\texttt{0x03}\|V)$. The input $m = \texttt{0x03}\|V$ is 56 bytes. After padding, it occupies two 512-bit blocks. The second block contains only zero-padding and the 64-bit length field `0x000001C0` ($=448$), carrying no information about V. In the first block, the message words are

$$W_0^{(2)} = \texttt{0x03}\|V[0{:}3], \qquad W_i^{(2)} = V[4i{-}1\,{:}\,4i{+}3], \quad i = 1, \ldots, 13, \tag{16}$$

with $W_{14}^{(2)} = \texttt{0x80000000}$ and $W_{15}^{(2)} = 0$. The schedule computes $\sigma_0\big(W_i^{(2)}\big)$ for $i = 1, \ldots, 13$: 13 evaluations on slices of V at a 1-byte offset relative to compression 1.

Coverage. The two compressions together provide **26 relevant evaluations** of σ_0 per `generate` call. Within one compression, the words $W_1, \ldots, W_{13}$ are non-overlapping 4-byte slices, so each byte of V appears in at most one σ_0 input per compression. Let $c_j(b) \in \{0, 1\}$, for $j \in \{1, 2\}$, indicate whether byte $V[b]$ is contained in some σ_0 input of compression j. The total coverage is

$$c_1(b) + c_2(b) = \begin{cases} 0 & \text{if } b \in \{0, 1, 2\}, \\ 1 & \text{if } b = 3, \\ 2 & \text{if } b \in \{4, \ldots, 54\}. \end{cases} \tag{17}$$

Bytes $V[0], V[1], V[2]$ appear only in $W_0^{(1)}$ and $W_0^{(2)}$, to which σ_0 is never applied (the schedule starts at W_1). These 24 bits constitute a *blind spot*, recovered separately via the round-0 T_1 leakage (Section 4.4). Byte $V[3]$ is covered once, by $W_1^{(2)} = V[3{:}7]$. All remaining bytes $V[4]$ through $V[54]$ are covered twice, once per compression, at different positions within the 32-bit word.

Edge cases. The analysis above considers the minimal case: a single 256-bit output with no additional input. When a `generate` call requests more than 256 bits, `Hashgen` evaluates $\texttt{SHA-256}(V{+}1)$, $\texttt{SHA-256}(V{+}2)$, …, each providing 26 additional σ_0 evaluations on inputs that differ from V by at most a few bits. Furthermore, when additional input is provided (Section 10.1.1.4 of [BK15]), the `generate` call first computes $w = \texttt{SHA-256}(\texttt{0x02}\|V\|\mathit{addin})$ and replaces V by $(V + w) \bmod 2^{440}$. This updated V is then used for the two main compressions. The σ_0 attack recovers the modified state regardless, which implies that Exp-SR is unaffected. Exp-PR requires the attacker to know the additional input to reconstruct the state update, but NIST SP 800-90A does not treat this input as secret [BK15], rendering Exp-PR equally feasible. In both cases, the attacker gains strictly more σ_0 observations than in the minimal case, which strictly reinforces the attack described above.

[2]Regardless of the requested output length, the first `Hashgen` invocation is always $\texttt{SHA-256}(V)$.

4.2 Per-Bit Leakage and the Narrow-Classifier Ceiling

In any software implementation, σ_0 is decomposed into a sequence of register-level shift, rotate, and XOR operations. Under the HW leakage model (Section 2.3), each register write leaks a noisy observation of the written value's Hamming weight. A binary classifier targeting bit j of the 32-bit σ_0 input observes the trace $\mathbf{t}$ and outputs the Log-Likelihood Ratio (LLR)

$$\Lambda_j = \log \frac{P(x_j = 1 \mid \mathbf{t})}{P(x_j = 0 \mid \mathbf{t})}. \tag{18}$$

A classifier restricted to a narrow window around a single instruction observes $\mathrm{HW}(r) + \mathcal{N}(0, \sigma^2)$, where r is an n-bit register containing the target bit j and $(n-1)$ other bits that, for uniformly random inputs, are approximately independent and uniformly distributed. Bit j contributes unit amplitude to the sum. The remaining bits add variance $(n-1)/4$. For $n = 8$ (the narrowest register width we evaluate), the per-bit signal-to-noise ratio is bounded by

$$\mathrm{SNR}_{\mathrm{bit}} = \frac{1}{\sigma^2_{\mathrm{meas}} + (n-1)/4} \;\leq\; \frac{4}{n-1}. \tag{19}$$

At $n = 8$, this yields $\mathrm{SNR}_{\mathrm{bit}} \leq 4/7 \approx 0.57$, corresponding to a maximum achievable accuracy of $\sim$64% under the exact Bayesian optimum for a single noiseless Hamming-weight observation. The Bayes-optimal accuracy of a single-observation bit classifier follows from modelling $\mathrm{HW}(r)$ as Gaussian. The target bit shifts the mean by one, while the remaining bits and measurement noise contribute the variance in the denominator of Eq. (19). The optimal accuracy is then $\Phi\big(\frac{1}{2}\sqrt{\mathrm{SNR}_{\mathrm{bit}}}\big)$, where Φ is the standard normal CDF. At $\mathrm{SNR}_{\mathrm{bit}} = 4/7$ this reproduces the bound stated above. Wider registers only worsen this ceiling, rendering narrow-window classifiers insufficient for state recovery. This fundamental ceiling motivates the wide-field classifier (cf. Section 4.5), which aggregates leakage from *all* instructions in a single forward pass.

We validate this empirically on the XMEGA target. A classifier trained on a single isolated instruction (NOP-padded, known input) achieves 65.9% mean accuracy (min 54.2%, max 77.0%), closely matching the theoretical $\sim$64% bound, with the small excess attributable to leakage beyond the pure HW model. A narrow-window CNN (250 samples, $\sim$1.5 instructions, centered on each bit's peak leakage sample) trained on 200k windows reaches 71.6% mean accuracy (min 61.6%, max 89.9% for a single outlier bit). We attribute the higher accuracy to the window capturing multiple register writes and to leakage beyond the pure HW model. In both cases, increasing the training set beyond 100k samples does not improve accuracy, confirming that the ceilings are fundamental rather than data-driven. While both baselines exceed the theoretical single-instruction bound, they remain well below the 80% per-compression accuracy the attack requires at minimum (cf. Section 4.3).

We quantify the leakage of each input bit using the Test Vector Leakage Assessment (TVLA), which applies Welch's t-test to detect data-dependent leakage in the power traces. A bit is considered to leak exploitable information at a sample point when the magnitude of its t-statistic exceeds the standard threshold of 4.5 w.r.t. a sensitive random variable. Figure 2 confirms that every input bit produces statistically significant leakage ($|t| \gg 4.5$) across thousands of sample points in the σ_0 power trace. While the leakage intensity varies across the trace, virtually every region of the σ_0 computation exceeds the $|t| = 4.5$ significance threshold, providing a wide-field classifier with exploitable signal at nearly every sample point.

4.3 Two-Compression Combining

Even at high per-bit accuracy, full-state recovery from a *single* observation remains unlikely. To illustrate, consider a hypothetical per-bit accuracy of $p = 0.99$: the probability of at

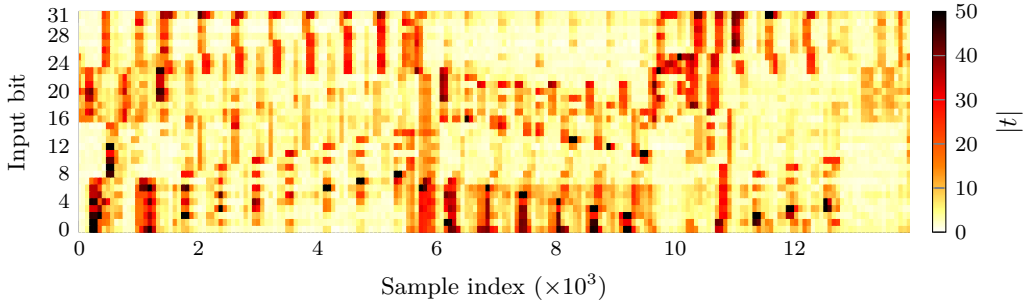


Figure 2: Bit-partitioned TVLA on the XMEGA target (10k traces). Each row shows the $|t|$-statistic for traces partitioned by one input bit ($j = 0, \ldots, 31$). All 32 bits exhibit significant leakage ($|t| \gg 4.5$) distributed across the full σ_0 trace, confirming that a wide-field classifier can aggregate information from many intermediates.

least one bit error among the 416 covered bits is $1 - 0.99^{416} \approx 98.5\%$. Moreover, the probability of failing Exp-PR (Algorithm 2), which requires two consecutive error-free recoveries, would be $1-(0.99^{416})^2 \approx 99.98\%$. While an adversary would eventually succeed given enough `generate` observations, this would render the technique less practical.

This problem is particularly severe when the DRBG output is not observed (pub = false): without brute-force verification, every bit must be recovered accurately. We resolve this issue by exploiting the two-compression structure of Hash-DRBG's `generate` call. By combining LLRs, we achieve a drastic reduction of the per-call failure rates. Assuming the aforementioned $p = 0.99$, the failure rate for Exp-SR decreases from 98.5% to 4.1%, whereas the failure rate for Exp-PR decreases from 99.98% to only 8.0% under the independence assumption.

NIST's Hash-DRBG grants us multiple, independent observations by specification: for $V[4]$ through $V[54]$, each bit appears as an input to two σ_0 evaluations given by Eq. (17), one from each compression, executed at different times and on different register contents. The two compressions execute on different register contents at different points in the trace, sharing no state beyond V itself. The per-bit LLRs are therefore independent and may be summed:

$$\Lambda_j^{\text{comb}} = \Lambda_j^{(1)} + \Lambda_j^{(2)}. \tag{20}$$

Crucially, the `0x03` prefix in compression 2 shifts the byte alignment by one position: where compression 1 processes $W_i^{(1)} = V[4i{:}4i{+}4]$, compression 2 processes $W_i^{(2)} = V[4i{-}1{:}4i{+}3]$. As a result, the same V bit appears at *different bit positions* within the σ_0 input word in the two compressions, and the two LLR contributions $\Lambda_j^{(1)}$, $\Lambda_j^{(2)}$ originate from *different classifiers* with potentially different accuracies p_1, p_2. Under a uniform prior on x_j, a classifier of accuracy p assigns to its favoured value the calibrated log-odds $\text{logit}(p) = \log \frac{p}{1-p}$, since a binary decision correct with probability p carries likelihood ratio $\frac{p}{1-p}$. Because the two compressions are conditionally independent given x_j, their log-odds add as in Eq. (20), so when both favour the correct value the combined log-odds equal $\text{logit}(p_1) + \text{logit}(p_2)$. Applying the logistic link $p = 1/(1 + e^{-\Lambda})$ to this sum yields the combined accuracy

$$p_{\text{comb}} = \frac{1}{1 + e^{-(\text{logit}(p_1)+\text{logit}(p_2))}}. \tag{21}$$

When $p_1 = p_2 = p$, this simplifies to $p_{\text{comb}} = 1/(1 + e^{-2\,\text{logit}(p)})$.

We verify the independence assumption empirically on the XMEGA target. Over held-out `generate` calls, the two compressions' per-bit LLRs are conditionally uncorrelated given the state bit, with a mean Pearson correlation of 0.007. The measured combined accuracy agrees with Eq. (21) to within 0.04 percentage points.

Table 1: Attack feasibility in the public-output setting ($\mathsf{pub} = \mathsf{true}$) vs. combined per-bit accuracy p_{comb} after two-compression LLR combining via Eq. (21). p: single-compression accuracy assuming $p_1 = p_2 = p$. Errors: expected over 416 covered bits. Cost: brute-force SHA-256 evaluations. Time assumes 10^9 SHA-256 evaluations per second on a commodity GPU [Ste24]. In the blind setting ($\mathsf{pub} = \mathsf{false}$), no errors can be tolerated.

p	p_{comb}	Errors	$\log_2$ cost	Time
50%	50.0%	208.0	208	10^{46} yr
60%	69.2%	128.0	128	10^{22} yr
70%	84.5%	64.6	65	1200 yr
75%	90.0%	41.6	42	1.2 h
80%	**94.1%**	**24.5**	**25**	**34 ms**
84%	96.5%	14.6	15	33 μs
88%	98.2%	7.6	8	0.3 μs
92%	99.2%	3.1	3	8 ns
96%	99.8%	0.7	1	2 ns
100%	100.0%	0.0	0	instant

When the random output is observed ($\mathsf{pub} = \mathsf{true}$), residual errors can be corrected by brute-force enumeration over the least-confident bits (cf. Section 4.6), verified against the observed DRBG output. At $\sim 10^9$ evaluations per second on a modern GPU [Ste24], we classify attacks with $\leq 2^{25}$ evaluations as *practical* (under one second) and those with $\leq 2^5$ evaluations as *trivial* (instantaneous). The practical threshold lies at $\boldsymbol{p_{\text{comb}} \geq 94\%}$, corresponding to $\sim$80% per single compression and yielding $\sim$25 expected errors over 416 covered bits (Table 1). Below $p_{\text{comb}} \approx 84\%$, the search space exceeds 2^{65} and the attack becomes infeasible. Above $p_{\text{comb}} \approx 99\%$, fewer than 4 errors remain and the attack is trivial.

4.4 Tractable vs. Intractable Leakage

Beyond the message schedule, SHA-256's compression function also processes V through the round equations in Eqs. (7)–(10). We briefly characterize which of these operations are exploitable.

Round-0 leakage. At round 0, the working variables equal the public SHA-256 IV. The round adds W_0 to known constants:

$$T_1 = \underbrace{h_{\text{IV}} + \Sigma_1(e_{\text{IV}}) + \text{Ch}(e_{\text{IV}}, f_{\text{IV}}, g_{\text{IV}}) + K_0}_{\text{known constant}} + W_0. \tag{22}$$

Since W_0 is the sole unknown, an attacker may target any intermediate computation for which W_0 is the only unknown input — most naturally, T_1 itself. Crucially, this applies to *both* compressions: $W_0^{(1)} = V[0{:}4]$ and $W_0^{(2)} = \texttt{0x03} \| V[0{:}3]$. Both are processed at round 0 with the same public IV, providing two independent observations per `generate` call for bytes $V[0], V[1], V[2]$ (the σ_0 blind spot). Even without exploiting this leakage, the 24 blind-spot bits can be brute-forced in the public-output setting at a cost of $2^{24} \approx 16$M SHA-256 evaluations, negligible on a modern GPU [Ste24]. Training per-bit classifiers on the round-0 computation trace can further reduce or eliminate this cost (cf. Section 6).

Rounds $\geq$ 1: self-protecting. From round 1 onward, each T_1 involves W_t added to five quantities that depend on all prior message words $W_0, \ldots, W_{t-1}$. The modular addition mixes the unknown prior state with W_t through carry propagation, destroying the direct relationship between W_t and the observable intermediates.

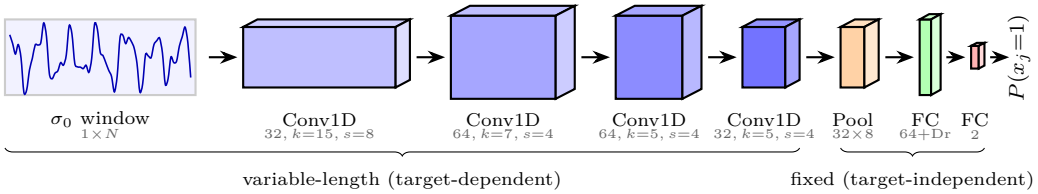


Figure 3: Wide-field strided CNN architecture. One model is trained per input bit ($j = 0, \ldots, 31$). Cuboid width represents temporal dimension (shrinking with strides), height represents channel count. Each convolutional block includes batch normalization [IS15] and ReLU activation. Four strided convolutions progressively reduce the full σ_0 window. `AdaptiveAvgPool1D` normalizes variable input lengths to a fixed 32×8 feature map, allowing the same architecture to be applied across targets of different register widths.

Summary. The identified exploitable leakage from a single `generate` call consists of:

1. **Message-schedule σ_0:** 26 relevant evaluations (13 per compression), covering $V[3{:}55]$ with one or two independent observations per byte.

2. **Round-0 T_1:** $W_0^{(1)} = V[0{:}4]$ and $W_0^{(2)} = \texttt{0x03} \| V[0{:}3]$, providing two independent observations of the σ_0 blind spot $V[0{:}3]$.

Together, every byte of V receives at least two independent observations per `generate` call (σ_0 for $V[4{:}55]$, round-0 T_1 for $V[0{:}3]$, both mechanisms for $V[3]$). All remaining SHA-256 operations are self-protecting through nonlinear mixing of unknown state.

4.5 Wide-Field Strided CNN

The narrow-classifier ceiling of Section 4.2 arises because a short trace window around one instruction captures only a single register write, where the target bit is buried under $(n-1)$ co-resident bits. However, the full σ_0 computation spans thousands of samples and involves many register writes, each depending on the same input bit through different intermediate expressions. A classifier that observes the *entire* σ_0 trace (cf. Figure 2) can learn to combine these multiple leakage points in a single forward pass, effectively circumventing the per-instruction ceiling.

We train one binary CNN per input bit ($j = 0, \ldots, 31$), each receiving the full σ_0 power trace as input and predicting whether bit j of the 32-bit input word is 0 or 1.

Handling variable trace lengths. The duration of a σ_0 computation depends on the target architecture: on the 8-bit and 16-bit targets (MC8051, MSP430, XMEGA), iterative shift loops produce windows of 12,000–27,000 samples, while on the 32-bit RV32I, native 32-bit operations reduce execution to ∼6,000 samples. Rather than designing a separate architecture per target, we use a single CNN with an `AdaptiveAvgPool1D` layer that maps any input length to a fixed-size feature vector of $32 \times 8 = 256$ values. The convolutional layers use identical kernel sizes and strides across all targets. Only the input dimension varies.

Figure 3 illustrates the architecture.

Each model has ∼63k parameters. The aggressive strides in the early layers act as learned downsampling: the first layer's stride of 8 roughly corresponds to one feature per clock-cycle group, while deeper layers combine information across instructions. Training uses binary cross-entropy loss with the AdamW optimizer (learning rate 10^{-3}, cosine annealing, up to 150 epochs with early stopping at patience 20, batch size 128). We

selected these hyperparameters manually on a held-out validation set, guided by established CNN-based SCA architectures [ZBHV20, WAGP20]. We did not perform an exhaustive hyperparameter search.

Unlike narrow per-instruction classifiers, this architecture is not limited by the single-register HW ceiling. It can exploit correlations across instructions. For instance, the same input bit may contribute to a byte load, a shift intermediate, and a final XOR result at different trace positions, enabling an implicit multi-point combination achieving $\gg$70% per-bit accuracy for all bits.

4.6 Recovery Procedure

Given a single `generate` trace, the recovery proceeds in four steps.

Step 0: Preprocessing. The captured power trace is z-score normalized (per-sample mean subtraction and standard deviation scaling) using the statistics of the training set, ensuring that the input distribution matches the one the classifiers were trained on.

Step 1: Per-word classification. For each of the 26 relevant σ_0 evaluations (13 per compression), the corresponding trace segment is fed to all 32 input-bit classifiers. Each classifier outputs an LLR Λ_j from Eq. (18) for its target bit. This yields a 32-dimensional LLR vector per σ_0 word.

Step 2: V-bit aggregation. The per-word LLRs are mapped to per-V-bit LLRs using the known byte alignments in Eqs. (15)–(16). For $V[4]$ through $V[54]$, each bit receives LLR contributions from two σ_0 evaluations (one per compression), which are summed as in Eq. (20). For $V[3]$, the σ_0 LLR from compression 2 can be combined with the T_1 LLR from compression 1, as both provide independent observations of $V[3]$. Bits in the blind spot $V[0{:}3]$ receive no σ_0 LLR but are recovered via the round-0 T_1 classifiers (Section 4.4).

Step 3: State completion. The maximum-likelihood estimate $\hat{V}$ is obtained by thresholding each LLR at zero: $\hat{V}_j = \mathbf{1}[\Lambda_j^{\text{comb}} > 0]$ for each covered bit. The estimate contains residual bit errors on the 416 covered bits and leaves the 24 blind-spot bits $V[0{:}3]$ undetermined. The total search space for brute-force correction is 2^{E+B}, where E is the number of covered-bit errors and B is the number of unknown blind-spot bits.

In the public-output setting ($\mathsf{pub} = \mathsf{true}$), candidates are verified against the observed DRBG output ($\texttt{Hashgen}(V_{\text{cand}}) = \textit{output}$). When the covered-bit Bit Error Rate (BER) is low ($E \lesssim 25$), the total search space is $\leq 2^{24+E}$, feasible on a modern GPU [Ste24]. When the covered-bit BER is high, the combined search space may exceed practical limits. In this case, the attacker additionally targets the round-0 W_0 leakage (Section 4.4) by training per-bit classifiers on the round-0 computation trace, reducing or eliminating the 24 blind-spot unknowns and bringing the total back into the practical regime.

In the blind setting ($\mathsf{pub} = \mathsf{false}$), brute-force verification is not available (Section 3). The attack then requires near-deterministic recovery ($E = 0$, $B = 0$), achievable by using additional T_1 classifiers.

4.7 Prediction Resistance Break

Recovering V from a single trace already compromises the current invocation: the attacker can compute $\texttt{SHA-256}(V)$ and derive any secret generated from that call (e.g., a session key). However, a single recovery does not suffice to predict *future* outputs. The state update $V' = V + \texttt{SHA-256}(\texttt{0x03} \| V) + C + ctr \pmod{2^{440}}$ depends on the unknown constant C, derived from V during instantiation.

Given two consecutive recoveries V_1 (trace 1) and V_2 (trace 2, with no intervening reseed), the attacker computes

$$\delta = \big(V_2 - V_1 - \texttt{SHA-256}(\texttt{0x03}\|V_1)\big) \bmod 2^{440} = C + ctr_1. \tag{23}$$

The individual values of C and ctr_1 need not be separated. δ alone suffices. Since ctr increments by 1 per `generate` call, all future states are determined:

$$V_{n+1} = \big(V_n + \texttt{SHA-256}(\texttt{0x03}\|V_n) + \delta + (n-1)\big) \bmod 2^{440}. \tag{24}$$

This constitutes a **complete break of prediction resistance** (Exp-PR, Algorithm 2): the attacker can compute $\texttt{SHA-256}(V_n)$ for every future `generate` call until the next reseed.

Avalanche prevents multi-trace improvement. A single-bit error in V_1 causes the hash $\texttt{SHA-256}(\texttt{0x03}\|V_1)$ to differ from the correct value in approximately half its bits, rendering δ and all subsequent predictions incorrect. Multi-trace consistency checking therefore requires *exact* recovery of at least two consecutive states. In our attack, residual bit errors are resolved by brute-force enumeration when the BER is non-zero (Section 4.6). The attacker is therefore fundamentally limited to single-trace recovery. Multi-trace strategies cannot improve upon it. Note that the publicly observed output $y = \texttt{Hashgen}(V)$ does not provide additional partial information about V beyond verification: SHA-256's preimage resistance prevents backward inference from y to V. Conversely, a countermeasure that defeats single-trace recovery is sufficient to prevent the prediction resistance break (cf. Section 7).

5 Experimental Setup

We evaluate the attack on four microcontroller architectures spanning register widths of 8, 16, and 32 bits, all running the same Hash-DRBG firmware.

5.1 Target Implementation

We use the ANSSI-FR `libdrbg` library [ANS22], an open-source C implementation of NIST SP 800-90A DRBGs. We target the Hash-DRBG instantiation with SHA-256. The implementation is unprotected: no masking, shuffling, or other SCA countermeasures are applied. This represents the standard deployment scenario, as justified in Section 3. The choice of implementation is not critical: since our attack targets the NIST specification rather than implementation-specific behavior, any compliant unprotected software implementation is potentially vulnerable. Following best practices, we wrap the library in a UART-based firmware interface that asserts a GPIO trigger on each `generate` call. Cross-correlation against an initially identified round reference pattern then locates the individual σ_0 windows within the captured trace.

We evaluate the attack on four targets: an MC8051 (8-bit), MSP430 (16-bit), and RV32I (32-bit) synthesized as soft cores on a NewAE CW305 (Artix 7) FPGA board, and an ATxmega128D4 (8-bit AVR hard core) on a NewAE CW308 baseboard. The three FPGA targets share the same clock frequency, power measurement point, and capture configuration, with register width as the primary differentiating factor.

5.2 Measurement Setup

Figure 4 shows the measurement configuration. A shunt resistor on the target's supply rail is sampled by a PicoScope 5000A at 12-bit resolution, 625 MS/s, with 20 MHz bandwidth limit. Each acquisition captures the full `generate` call (both SHA-256 compressions), triggered by a GPIO pulse.

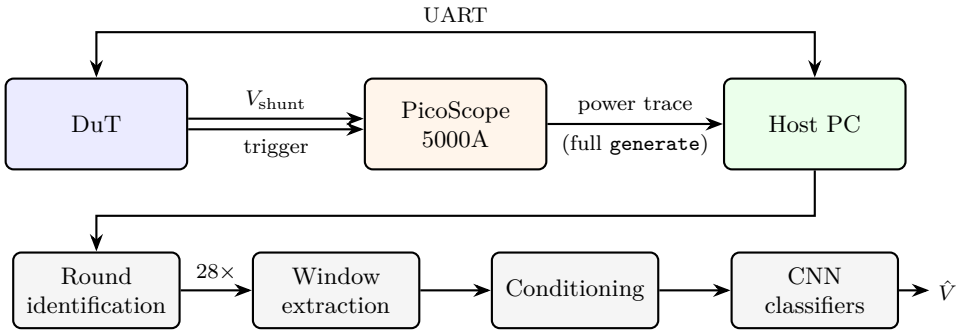


Figure 4: Measurement and processing pipeline. The DuT executes a Hash-DRBG `generate` call while the PicoScope digitizes the power trace via a shunt resistor. A cross-correlation-based round identification step locates the 26 relevant σ_0 evaluations and 2 round-0 windows within the full trace. The corresponding 28 windows are extracted, conditioned (z-score normalization), and fed to the per-bit CNN classifiers to recover $\hat{V}$.

5.3 Capture and Trace Extraction

During profiling, the oscilloscope captures the power trace of an entire `generate` call, encompassing both SHA-256 compressions, in a single acquisition. A full `generate` trace contains 192 σ instances (96 per compression), of which only the 26 σ_0 evaluations on the direct message words $W_1, \ldots, W_{13}$ are exploitable (Section 4.1). Since all 192 evaluations produce near-identical power patterns (shift/rotate loops), extraction requires two steps: first, cross-correlation against a manually identified reference pattern locates all σ instances (σ_0 and σ_1) in the trace. Second, the known execution order, derived from firmware analysis, is used to count and select the 26 target σ_0 windows by their position in the fixed round sequence. The correct assignment is confirmed initially by leakage-detection t-tests partitioned by the known input word. This procedure automatically extracts all 26 target windows per `generate` call without requiring individual firmware triggers per σ_0 evaluation. Each extracted window is z-score normalized (per-sample mean subtraction and standard deviation scaling across the training set) before being presented to the classifiers. Between `generate` calls, the state is updated via the DRBG transition function, maintaining a realistic state chain.

The profiling dataset for each target consists of up to 500,000 σ_0 windows (at 26 windows per `generate` call), split into training and validation sets. The number of profiling windows varies per target. We evaluate all targets at multiple training set sizes to characterize scaling behavior (Figure 5). All data is stored in HDF5 format.

Training all 32 σ_0 classifiers takes approximately 30 minutes at 5k windows and 24 hours at 500k windows on an NVIDIA RTX 5080 GPU (16 GB VRAM). The T_1 classifiers converge faster, training in roughly half the time. Since all 64 classifiers (32 σ_0 + 32 T_1) are independent, training is trivially parallelizable.[3]

6 Results

We evaluate the σ_0 attack on multiple architectures exhibiting different register widths, all executing identical Hash-DRBG firmware (Section 5). Of the full 440-bit state V, the σ_0 classifiers recover up to 416 bits via two-compression LLR combining (Section 4.3), with the remaining 24 blind-spot bits handled as described in Section 4.4.

[3]The artifacts (source code, trained models, and trace datasets) will be made available at https://github.com/HWS-XMS/sigma0-hash-drbg.

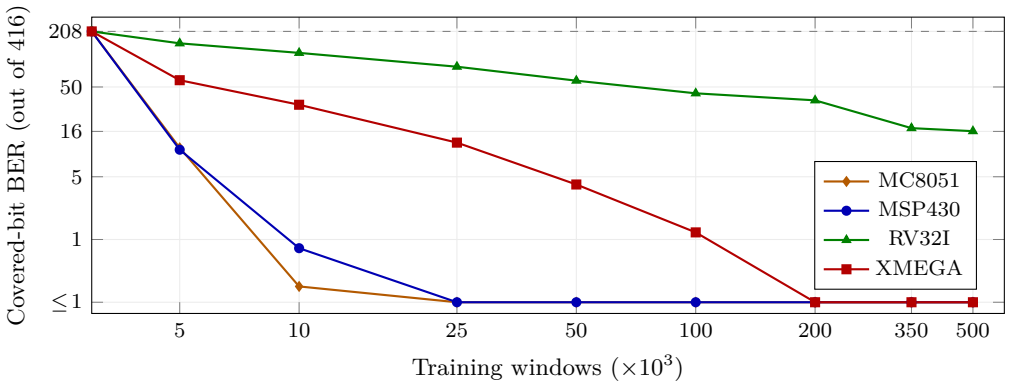


Figure 5: Covered-bit BER vs. training set size across four architectures (1,000 held-out calls each, log–log scale). Values at the floor (≤ 1) indicate zero covered-bit errors. The MC8051 and MSP430 converge within 10k–25k windows, the XMEGA at 200k. The RV32I plateaus at ∼16 (500k), approaching its architectural ceiling.

6.1 Classifier Performance

Figure 5 shows how the covered-bit BER scales with the number of profiling windows, evaluated on 1,000 held-out `generate` calls each.

The 8-bit MC8051 (soft core, CW305 FPGA) reaches 100% validation accuracy on all 32 bits at 50k windows. The 16-bit openMSP430 (soft core, same CW305 platform) decomposes σ_0 into long iterative shift sequences over 16-bit registers, producing many intermediates with strong per-bit leakage. Every bit reaches 100% and the BER drops to zero at 25k windows. The 32-bit RV32I (soft core, same CW305 platform) exhibits a fundamentally different profile: the wider datapath collapses σ_0 into fewer instructions with higher per-register noise (Section 4.2), leaving some bits with strong leakage (e.g., IN17 at 99.8%, where INk denotes input bit k of the 32-bit σ_0 word) while others barely exceed the narrow-classifier ceiling. At 86.2% mean accuracy (500k windows), the BER plateaus at ∼16 covered errors. This is still within the practical regime of Table 1, but far from the deterministic recovery achieved on the 8- and 16-bit targets. The 8-bit XMEGA (hard core, CW308 baseboard) follows a trajectory between the MC8051 and the MSP430: 99.1% mean accuracy at 200k windows, with zero covered-bit errors from 200k windows onward.

6.2 State Recovery (Exp-SR)

Two-compression LLR combining (Section 4.3) provides two independent observations for each covered byte of V. The combined effective accuracy exceeds the practical threshold of Table 1 for the pub = true setting on all evaluated targets.

On the MC8051, MSP430, and XMEGA, all 416 covered bits are recovered perfectly, with zero bit errors on every held-out call (Figure 5). On the RV32I, the mean covered-bit BER is ∼16 at 500k windows (min 6, max 28).

The σ_0 attack leaves 24 blind-spot bits $V[0{:}3]$ uncovered (Section 4.4). These are comfortably brute-forceable at 2^{24} in the pub = true setting, assuming perfect recovery by our σ_0 attack. In case of imperfect σ_0 recovery or in the pub = false setting, we additionally train 32 per-bit classifiers targeting the round-0 intermediate T_1 (Section 4.4), using the same CNN architecture (Section 4.5). This recovers the missing W_0 bits and reduces or eliminates the blind-spot unknowns. Both compressions provide independent T_1 observations ($W_0^{(1)} = V[0{:}4]$ and $W_0^{(2)} = \texttt{0x03} \| V[0{:}3]$). The 0x03 prefix shifts the byte

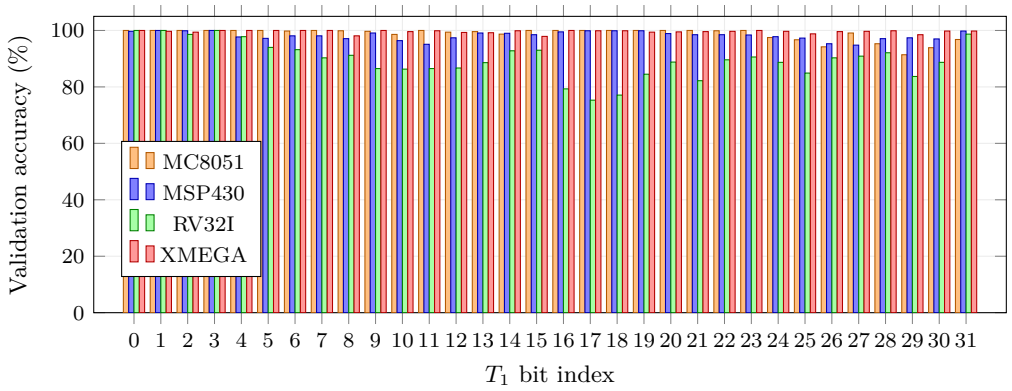


Figure 6: Per-bit round-0 T_1 classifier accuracy across architectures (MC8051 100k, MSP430 100k, RV32I 500k, XMEGA 200k training windows). The MC8051 (min 91.4%, max 100%, μ=98.8%), MSP430 (min 94.8%, max 100%, μ=98.2%), and XMEGA (min 97.9%, max 100%, μ=99.6%) perform reliably on every bit. The RV32I (min 75.3%, max 100%, μ=89.7%) still recovers every bit, but exhibits high variance: LSB positions reach deterministic accuracy, whereas mid-range bits drop to 75–90%. Two-compression combining pairs weak positions in one compression with stronger positions in the other.

Table 2: Round-0 T_1 recovery (V[0:4], 32 bits) BER (1,000 held-out calls each).

Training	MC8051 (8-bit)	MSP430 (16-bit)	RV32I (32-bit)	XMEGA (8-bit)
5k	9.1	8.3	10.6	4.7
10k	2.7	5.6	8.7	2.0
25k	0.2	1.8	7.5	1.0
50k	0.1	0.7	6.0	0.2
100k	0.0	0.2	4.6	0.0
200k	0.0	0.0	3.3	0.0
350k	0.0	0.0	2.4	0.0
500k	0.0	0.0	1.6	0.0

alignment so that each V byte appears at a different bit position in each compression's T_1 (Figure 6). On the MC8051 and XMEGA, two-compression combining yields perfect blind-spot recovery (zero BER) from 100k windows onward, and on the MSP430 from 200k windows (Table 2). On the RV32I, the byte-shift diversification ensures that each V byte pairs a weaker estimate with a stronger one, reducing the blind-spot BER to 1.6 at 500k windows. Note that the attacker knows which bits exhibit low classifier confidence from the LLR magnitudes, enabling targeted enumeration of the least-confident bits rather than exhaustive search.

6.3 Prediction Resistance Break (Exp-PR)

Given the per-call state recovery demonstrated above, winning Exp-PR (Algorithm 2) is immediate: two consecutive `generate` traces yield δ via Eq. (23), after which all future outputs follow from Eq. (24). On the MC8051 and MSP430, $\sigma_0 + T_1$ recovery is fully deterministic, whereas on the XMEGA, recovery is near-deterministic. In both cases, the prediction resistance break succeeds without any brute-force and is viable in the blind setting. On the RV32I (`pub` = `true` required, Section 3), the per-trace search space of $\sim 2^{18}$ must be exhausted twice, remaining well within practical limits. In the following, we evaluate the fully blind recovery on the hardest, yet viable, target, namely the XMEGA.

Table 3: Per-architecture attack summary ($\sigma_0 + T_1$ recovery).

Target	Reg.	σ_0 windows	T_1 windows	$\log_2$ cost	pub=true	pub=false
MC8051	8-bit	50k	100k	0	Deterministic	Deterministic
MSP430	16-bit	100k	200k	0	Deterministic	Deterministic
RV32I	32-bit	500k	500k	$\sim$18	Practical	Infeasible
XMEGA	8-bit	200k	200k	$\leq$1	Trivial	Near-det. (99%)

XMEGA blind recovery. We select the XMEGA to validate Exp-SR and Exp-PR in the blind setting (pub = false, Algorithm 2). Despite achieving $\geq 99.6\%$ per-bit accuracy on every classifier, the theoretical analysis of Section 4.3 predicts that full 440-bit recovery will occasionally fail: at these accuracies, a non-zero fraction of calls will contain one or two bit errors across the 416 covered σ_0 bits. In the pub = true setting, such errors are inconsequential: at most 1 bit error was observed on any call, yielding a trivial search space.

For the blind setting, we evaluate on 1,000 held-out calls each for two scenarios: In the *same-device* setting (profiling *and* attack on the same XMEGA board), state recovery succeeds on 990/1,000 calls (99.0%) and prediction resistance breaks on 490/500 pairs (98.0%). In the *cross-device* setting (classifiers trained on one board, transferred to an attacked one), state recovery succeeds on 972/1,000 calls (97.2%) and prediction resistance breaks on 473/500 pairs (94.6%). All 95% confidence intervals are within ± 2 percentage points. The T_1 classifiers produce zero errors on the same device. All 10 same-device failures are single σ_0 bit errors. The cross-device gap is driven by T_1 degradation rather than σ_0, which is nearly identical across devices. This constitutes a complete, experimentally validated break of the NIST prediction resistance property under the most restrictive attacker model: no output oracle, no device-specific profiling. *Any* single bit error across the 32 T_1 and 13×32 σ_0 classifications fails the entire attack.

Table 3 summarizes the end-to-end attack across all targets and settings. On the MC8051, MSP430, and XMEGA, $\sigma_0 + T_1$ recovery is *deterministic* or *near-deterministic*: perfect recovery on MC8051 and MSP430, near-perfect on XMEGA. On the RV32I, the total search space is $\sim 2^{18}$, comfortably practical in the pub = true setting.

7 Countermeasures

Our attack exploits exactly two leakage surfaces within SHA-256, the σ_0 evaluations in the message schedule and the round-0 leakage of W_0 (Section 4.4). We show that the σ_0 surface can be provably eliminated, mitigating the attack.

7.1 Why Targeted Masking Suffices

SHA-256 is awkward to mask because its compression function mixes two kinds of operation. The Σ, σ, Ch, and Maj functions are Boolean, built from XOR, AND, rotation, and shift, whereas the round temporaries and working-variable updates use addition modulo 2^{32}. Boolean masking is cheap for the first and arithmetic masking natural for the second, but a value cannot cross between the two domains for free. Each crossing requires a secure mask conversion [Gou01, Cor17]. Due to SHA-256 internals, a fully masked implementation pays this conversion cost hundreds of times per invocation. Due to this fact, we provide a targeted fix tailored to our attack. For highest security assurance, we recommend masking the full SHA-256. As established in Section 4.7, SHA-256's avalanche effect limits the attacker to single-trace recovery, hence protecting σ_0 and round-0 W_0 against single-trace extraction is sufficient.

7.2 Masking σ_0

The key observation is that σ_0 is GF(2)-linear, as shown in Eq. (12). For any x and random mask $r \in \{0,1\}^{32}$,

$$\sigma_0(x) = \sigma_0(x \oplus r) \oplus \sigma_0(r). \tag{25}$$

Boolean masking of σ_0 is therefore exact. Evaluating $\sigma_0(W_i \oplus r)$ produces a computation whose every intermediate is statistically independent of W_i. The result is a pair of Boolean shares $\big(\sigma_0(W_i \oplus r),\ \sigma_0(r)\big)$ whose XOR equals $\sigma_0(W_i)$. We never recombine these shares, since forming $\sigma_0(W_i)$ in the clear would expose it in a single register write. Instead we transform the masking from the Boolean to the arithmetic domain.

We convert the two Boolean shares to arithmetic shares using the t-SNI variant of Goubin's first-order Boolean-to-arithmetic conversion [Gou01, Cor17]. The conversion produces (A_1, A_2) with $\sigma_0(W_i) = A_1 + A_2 \pmod{2^{32}}$, and both shares are consumed directly in the message-schedule addition, which becomes

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + A_1 + A_2 + W_{t-16} \pmod{2^{32}}. \tag{26}$$

The conversion is proven t-SNI secure in the ISW probing model [ISW03, Cor17]. Any single probed intermediate, even combined with an output share, can be perfectly simulated from one input share alone, ensuring that $\sigma_0(W_i)$ is never recoverable from a single probe.

This targeted countermeasure eliminates σ_0 leakage but does not protect the round-0 T_1 computation, which an adversary may still exploit (Section 4.4) to recover the four bytes $V[0{:}4]$. Even at deterministic recovery of these 32 bits, however, the remaining 408 bits of V are protected by the σ_0 masking and cannot be recovered from a single trace. Brute-forcing 2^{408} candidates is computationally infeasible, and SHA-256's avalanche effect (Section 4.7) prevents any cross-trace reduction. The single residual side channel that remains in the message schedule is the unmasked W_i load preceding the masking XOR, a single-instruction Hamming-weight observation bounded by the ∼65% ceiling of Section 4.2[4], far below the 94% threshold required by the attack (Table 1).

On our XMEGA target, masking one σ_0 evaluation costs 131 instructions against 64 for the unprotected version, an overhead of 105%. The Boolean-to-arithmetic conversion adds a further 65 instructions per word, and the masked message-schedule addition adds one more, since Eq. (26) has five terms instead of four. Across the 48 message-schedule evaluations, with σ_0 accounting for roughly one eighth of a SHA-256 compression, the complete countermeasure adds about 27% per SHA-256 invocation, and a `generate` call performs two such invocations. Crucially, our countermeasure crosses the Boolean-arithmetic boundary only 48 times per SHA-256, once per masked word, whereas a fully masked implementation would convert at every one of its ∼590 modular additions. Such an implementation is not known to exist in software, and it would additionally protect the nonlinear Ch and Maj functions through secure-AND gadgets [ISW03], at an estimated cost of several times the hash itself. Should stronger guarantees be required, the scheme generalizes to d shares by applying σ_0 independently to each share, with cost scaling linearly in d. A fully masked implementation would additionally eliminate the residual single-instruction W_i-load leakage. We leave it to future work, as our targeted countermeasure already defeats the presented attack.

7.3 Experimental Validation

We implement first-order masked σ_0 on all four targets (MC8051, MSP430, RV32I, XMEGA) and capture 10^6 windows per target. Bitwise TVLA confirms the absence of first-order leakage on both input and output bits of σ_0.

[4]The $n = 8$ ceiling. Wider registers ($n = 16, 32$) yield strictly lower ceilings.

We train the same wide-field strided CNN that achieves >98% per-bit accuracy on unmasked traces (Figure 5). On masked traces, all 32 input-bit classifiers converge to 49–51% validation accuracy on all four targets, consistent with random guessing. Although deep-learning or dissection attacks [MPP16, BS20] may recover marginal accuracy beyond 50%, this remains far below the $p_{\text{comb}} \geq 94\%$ that the attack requires (Table 1).

Combining the targeted masking with SHA-256's avalanche property yields a complete defense. σ_0 leakage is eliminated via t-SNI-secure Boolean-to-arithmetic conversion (validated at 49–51% accuracy on all targets), rounds ≥ 1 are self-protecting through nonlinear state mixing, and avalanche renders multi-trace accumulation infeasible. The attack identifies precisely which operations to protect, and the GF(2)-linearity of σ_0 enables low-overhead protection.

8 Conclusion

We have presented the first SCA on NIST SP 800-90A Hash-DRBG, a construction that, due to the absence of a persistent secret key, was previously considered resistant to side-channel attacks.

By targeting the σ_0 function in the SHA-256 message schedule, we showed that each `generate` call exposes 416 of the 440 state bits through 26 byte-shifted σ_0 evaluations. The remaining 24 blind-spot bits are recovered via brute-force (2^{24}), round-0 T_1 classifiers, or both. On all three 8- and 16-bit targets, recovery is either *deterministic* or *near-deterministic* from a single power trace. On the 32-bit RV32I, the search space remains practical ($\sim 2^{18}$), confirming register width as the dominant factor in attack effectiveness. Two consecutive state recoveries then yield the internal update constant, winning Exp-PR (Algorithm 2) and breaking the prediction resistance of Hash-DRBG entirely.

We derived and *experimentally validated* a targeted first-order Boolean masking countermeasure exploiting σ_0's GF(2)-linearity at low overhead. The attack thus identifies precisely which operations require protection, with first-order masking sufficing to drive all 32 σ_0 classifiers to random-guess accuracy across all four targets.

Limitations and future work. The attack is physical and single-trace, and its success depends on the observability setting of Section 3. When the DRBG output is available ($\mathsf{pub} = \mathsf{true}$), residual bit errors are corrected by brute-force enumeration and recovery is complete on all four targets. In the blind setting ($\mathsf{pub} = \mathsf{false}$), no errors can be tolerated, so the 32-bit RV32I with its $\sim 2^{18}$ residual does not yield the exact state required to break prediction resistance. Our targets include a commercial ASIC microcontroller, the ATxmega128D4, so these results are not artifacts of FPGA soft cores. Our attack targets software SHA-256, where the message schedule executes sequentially. Hardware accelerators that pipeline or fully unroll the rounds prevent per-instruction profiling, leaving their vulnerability an open question. While our masking reduces the CNN attack to random guessing, informed dissection strategies against masked implementations [BS20] may recover some accuracy, and whether this suffices to cross the feasibility threshold is left to future work. Beyond DRBGs, the σ_0 attack surface exists in any construction where SHA-256 processes secret data through the message schedule, most notably hash-based signature schemes such as SPHINCS+ and XMSS. Extending our single-trace methodology to these post-quantum constructions is a promising direction.

References

[ANS22] ANSSI-FR. libdrbg – NIST SP 800-90a DRBG implementations. https://github.com/ANSSI-FR/libdrbg, 2022.

[BBD+13] Sonia Belaïd, Luk Bettale, Emmanuelle Dottax, Laurie Genelle, and Franck Rondepierre. Differential power analysis of HMAC SHA-2 in the Hamming weight model. In *2013 International Conference on Security and Cryptography (SECRYPT)*, pages 1–12. IEEE, 2013.

[BCO04] Eric Brier, Christophe Clavier, and Francis Olivier. Correlation power analysis with a leakage model. In *Cryptographic Hardware and Embedded Systems – CHES 2004*, pages 16–29. Springer, 2004.

[BDT+21] Yaacov Belenky, Ira Dushar, Valery Teper, Hennadii Chernyshchyk, Leonid Azriel, and Yury Kreimer. First full-fledged side channel attack on HMAC-SHA-2. In *Constructive Side-Channel Analysis and Secure Design – COSADE 2021*, pages 31–52. Springer, 2021.

[BDT+23] Yaacov Belenky, Ira Dushar, Valery Teper, Vadim Bugaenko, Oleg Karavaev, Leonid Azriel, and Yury Kreimer. Carry-based differential power analysis (CDPA) and its application to attacking HMAC-SHA-2. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(3):1–29, 2023.

[BGK04] Johannes Blömer, Jorge Guajardo, and Volker Krummel. Provably secure masking of AES. In *Selected Areas in Cryptography – SAC 2004*, pages 69–83. Springer, 2004.

[BK15] Elaine B Barker and John M Kelsey. *Recommendation for Random Number Generation Using Deterministic Random Bit Generators (Revision 1)*. National Institute of Standards and Technology, 2015.

[BS20] Olivier Bronchain and François-Xavier Standaert. Side-channel countermeasures' dissection and the limits of closed source security evaluations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 1–25, 2020.

[CDP17] Eleonora Cagli, Cécile Dumas, and Emmanuel Prouff. Convolutional neural networks with data augmentation against jitter-based countermeasures: Profiling attacks without pre-processing. In *Cryptographic Hardware and Embedded Systems – CHES 2017*, pages 45–68. Springer, 2017.

[CKP+20] Shaanan Cohney, Andrew Kwong, Shahar Paz, Daniel Genkin, Nadia Heninger, Eyal Ronen, and Yuval Yarom. Pseudorandom black swans: Cache attacks on CTR_DRBG. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1241–1258. IEEE, 2020.

[Cor17] Jean-Sébastien Coron. High-order conversion from boolean to arithmetic masking. In *Cryptographic Hardware and Embedded Systems – CHES 2017*, pages 93–114. Springer, 2017.

[CRR02] Suresh Chari, Josyula R Rao, and Pankaj Rohatgi. Template attacks. In *Cryptographic Hardware and Embedded Systems – CHES 2002*, pages 13–28. Springer, 2002.

[DM20] Lauren De Meyer. Recovering the CTR_DRBG state in 256 traces. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 37–65, 2020.

[FLRV09] Pierre-Alain Fouque, Gaëtan Leurent, Denis Réal, and Frédéric Valette. Practical electromagnetic template attack on HMAC. In *Cryptographic Hardware and Embedded Systems – CHES 2009*, pages 66–80. Springer, 2009.

[Gou01] Louis Goubin. A sound method for switching between boolean and arithmetic masking. In *Cryptographic Hardware and Embedded Systems – CHES 2001*, pages 3–15. Springer, 2001.

[HS20] Viet Tung Hoang and Yaobin Shen. Security analysis of NIST CTR-DRBG. In *Advances in Cryptology – CRYPTO 2020*, pages 218–247. Springer, 2020.

[HWZ18] Zhenhao He, Liji Wu, and Xiangmin Zhang. High-speed pipeline design for HMAC of SHA-256 with masking scheme. In *2018 12th IEEE International Conference on Anti-counterfeiting, Security, and Identification (ASID)*, pages 174–178. IEEE, 2018.

[IS15] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*, pages 448–456. PMLR, 2015.

[ISW03] Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In *Advances in Cryptology – CRYPTO 2003*, pages 463–481. Springer, 2003.

[JXLW20] Xin Jin, Yong Xiao, Shiqi Li, and Suying Wang. Deep learning-based side channel attack on HMAC SM3. *IJIMAI*, 6(4):113–120, 2020.

[KJJ99] Paul Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *Advances in Cryptology – CRYPTO '99*, pages 388–397. Springer, 1999.

[KPH+19] Jaehun Kim, Stjepan Picek, Annelie Heuser, Shivam Bhasin, and Alan Hanjalic. Make some noise. Unleashing the power of convolutional neural networks for profiled side-channel analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 148–179, 2019.

[MOP07] Stefan Mangard, Elisabeth Oswald, and Thomas Popp. *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. Springer, 2007.

[Moz24] Mozilla. NSS FIPS module — Hash_DRBG implementation. https://github.com/nss-dev/nss/blob/master/lib/freebl/drbg.c, 2024.

[MPP16] Houssem Maghrebi, Thibault Portigliatti, and Emmanuel Prouff. Breaking cryptographic implementations using deep learning techniques. In *Security, Privacy, and Applied Cryptography Engineering – SPACE 2016*, pages 3–26. Springer, 2016.

[MTMM07] Robert McEvoy, Michael Tunstall, Colin C Murphy, and William P Marnane. Differential power analysis of HMAC based on SHA-2, and countermeasures. In *Information Security Applications – WISA 2007*, pages 317–332. Springer, 2007.

[OMPR05] Elisabeth Oswald, Stefan Mangard, Norbert Pramstaller, and Vincent Rijmen. A side-channel analysis resistant description of the AES S-Box. In *Fast Software Encryption – FSE 2005*, pages 413–423. Springer, 2005.

[Ope16] OpenJDK. JEP 273: DRBG-based SecureRandom implementations. https://openjdk.org/jeps/273, 2016.

[PCBP21] Guilherme Perin, Łukasz Chmielewski, Lejla Batina, and Stjepan Picek. Keep it unsupervised: Horizontal attacks meet deep learning. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 343–372, 2021.

[PPM+23] Stjepan Picek, Guilherme Perin, Luca Mariot, Lichao Wu, and Lejla Batina. SoK: Deep learning-based physical side-channel analysis. *ACM Computing Surveys*, 55(11):1–35, 2023.

[RP10] Matthieu Rivain and Emmanuel Prouff. Provably secure higher-order masking of AES. In *Cryptographic Hardware and Embedded Systems – CHES 2010*, pages 413–427. Springer, 2010.

[Sch22] Frank Schuhmacher. Canonical DPA attack on HMAC-SHA1/SHA2. In *Constructive Side-Channel Analysis and Secure Design – COSADE 2022*, pages 193–211. Springer, 2022.

[STBK+16] Meltem Sönmez Turan, Elaine Barker, John Kelsey, Kerry McKay, Mary Baish, and Michael Boyle. Recommendation for the entropy sources used for random bit generation. Technical report, National Institute of Standards and Technology, 2016.

[Ste24] Jens Steube. hashcat – advanced password recovery. https://hashcat.net/hashcat/, 2024.

[Tim19] Benjamin Timon. Non-profiled deep learning-based side-channel attacks with sensitivity analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 107–131, 2019.

[WAGP20] Lennert Wouters, Victor Arribas, Benedikt Gierlichs, and Bart Preneel. Revisiting a methodology for efficient CNN architectures in profiling attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 147–168, 2020.

[wol24] wolfSSL Inc. wolfssl embedded SSL/TLS library. https://www.wolfssl.com/, 2024.

[WS19] Joanne Woodage and Dan Shumow. An analysis of NIST SP 800-90a. In *Advances in Cryptology – EUROCRYPT 2019*, pages 151–180. Springer, 2019.

[YGS+17] Katherine Q Ye, Matthew Green, Naphat Sanguansin, Lennart Beringer, Adam Petcher, and Andrew W Appel. Verified correctness and security of mbedTLS HMAC-DRBG. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 2007–2020, 2017.

[YK20] Shih-Chun You and Markus G Kuhn. A template attack to reconstruct the input of SHA-3 on an 8-bit device. In *Constructive Side-Channel Analysis and Secure Design – COSADE 2020*, pages 25–42. Springer, 2020.

[YK21] Shih-Chun You and Markus G Kuhn. Single-trace fragment template attack on a 32-bit implementation of Keccak. In *Smart Card Research and Advanced Applications – CARDIS 2021*, pages 3–23. Springer, 2021.

[ZBHV20] Gabriel Zaid, Lilian Bossuet, Amaury Habrard, and Alexandre Venelli. Methodology for efficient CNN architectures in profiling attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020(1):1–36, 2020.

[ZXF+20] Libang Zhang, Xinpeng Xing, Junfeng Fan, Zongyue Wang, and Suying Wang. Multilabel deep learning-based side-channel attack. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(6):1207–1216, 2020.